Issue: Next.js Old Version Being Served After Deployment

We observed that after deploying a new version of the Next.js application, some users were still getting the previous version on their initial visit. A normal/hard refresh would then show the latest version.

Root Cause

The `/` HTML page was being statically prerendered by Next.js and was returning:

Cache-Control: s-maxage=31536000

`31536000` seconds equals **1 year**. This allowed the HTML response to remain fresh in shared caches for a very long time, which could result in users receiving an older version after deployment.

Fix Implemented on UAT

We added a cache policy specifically for the homepage:

```
async headers() {
  return [
    {
      source: "/",
      headers: [
        {
          key: "Cache-Control",
          value: "public, max-age=0, must-revalidate",
        },
      ],
    },
  ];
}
```

This ensures that the HTML response is revalidated instead of being considered fresh for one year.

Important Consideration

Initially, we applied the cache policy to `/:path*`. However, this also changed the caching behavior of Next.js static assets under `/_next/static/*`.

Therefore, the rule was narrowed to the `/` route only.

Current UAT Verification

- `/` → `max-age=0, must-revalidate`
- RSC response → `max-age=0, must-revalidate`
- `/_next/static/*.js` → `max-age=31536000, immutable`

The static JS caching should not be disabled because Next.js generates hashed filenames for these assets. When the application changes, new builds generate new asset URLs, allowing these files to be safely cached for a long period.

For Future Deployments

Whenever we deploy a new version of the Next.js application, we should:

1. Verify that the HTML/RSC responses are being revalidated.
2. Verify that `/_next/static/*` assets retain long-term immutable caching.
3. Perform a normal browser visit after deployment to confirm that the latest version is received without requiring a hard refresh.
4. Avoid applying HTML cache policies globally to all application paths and static assets.

UAT is currently fixed and verified at the HTTP header level. Production should be updated after completing the V1 → V2 normal-visit validation on UAT.